

VS Code AWS Infra and Build Guide

Connect VS Code to AWS, ECR, ECS and Terraform

Flipkart Clone Microservices Workspace

Link VS Code with AWS Infrastructure and Build Infra Guide

This guide explains how to connect your VS Code workspace to AWS, build Docker images, push them to ECR, deploy to ECS, and optionally create infrastructure from VS Code using Terraform.

Use this after your code is already opened in VS Code.

PART 1: What This Guide Helps You Do

You will learn how to:

- Connect VS Code terminal to your AWS account.
- Use AWS Toolkit extension in VS Code.
- Store AWS deployment variables in a local file.
- Build Docker images from VS Code.
- Push images to Amazon ECR.
- Force ECS services to deploy new images.
- Use Terraform from VS Code to build infrastructure if required.
- Connect GitHub Actions or CodePipeline for automated deployment.

PART 2: Install VS Code Extensions

Open VS Code extensions panel and install:

Extension	Purpose
AWS Toolkit	Connect VS Code with AWS account and resources
Docker	View and manage containers/images
HashiCorp Terraform	Terraform syntax and workflow
ESLint	JavaScript linting
Prettier	Code formatting
GitLens	Git history and blame
REST Client	Test APIs inside VS Code

The workspace also includes recommended extension settings in:

```
.vscode/extensions.json
```

PART 3: Configure AWS Access from VS Code Terminal

Option A: IAM Identity Center SSO Recommended

If your AWS account uses IAM Identity Center:

```
aws configure sso
```

Enter:

```
SSO session name: flipkart
SSO start URL: your AWS access portal URL
SSO region: us-east-1
CLI default client Region: us-east-1
CLI default output format: json
Profile name: flipkart-dev
```

Login:

```
aws sso login --profile flipkart-dev
```

Verify:

```
aws sts get-caller-identity --profile flipkart-dev
```

Option B: Access Keys

Use this only for development labs. Do not commit keys.

```
aws configure --profile flipkart-dev
```

Enter:

```
AWS Access Key ID
AWS Secret Access Key
Default region: us-east-1
Default output: json
```

Verify:

```
aws sts get-caller-identity --profile flipkart-dev
```

Make Profile Default in Current Terminal

Linux/macOS/Git Bash:

```
export AWS_PROFILE=flipkart-dev
export AWS_REGION=us-east-1
```

PowerShell:

```
$env:AWS_PROFILE="flipkart-dev"
$env:AWS_REGION="us-east-1"
```

Verify:

```
aws sts get-caller-identity
```


PART 4: Connect AWS Toolkit in VS Code

1. Open VS Code.
2. Click AWS icon in left sidebar.
3. Click **Connect to AWS**.
4. Choose your profile, for example `flipkart-dev`.
5. Select region `us-east-1`.
6. Use AWS Explorer to view resources.

Depending on extension version, you can view resources such as:

- ECR repositories
- ECS clusters and tasks
- CloudWatch Logs
- S3 buckets
- Lambda functions
- CloudFormation stacks

Use AWS Toolkit for quick navigation, but keep deployment commands scripted for repeatability.

PART 5: Create Local AWS Environment File

Create a local file from the template:

```
cp scripts/aws-env.example scripts/aws-env.local
```

Edit `scripts/aws-env.local`:

```
AWS_PROFILE=flipkart-dev  
AWS_REGION=us-east-1  
AWS_ACCOUNT_ID=123456789012  
ECS_CLUSTER=flipkart-cluster  
ALB_DNS_NAME=flipkart-alb-xxxx.us-east-1.elb.amazonaws.com
```

Never commit `scripts/aws-env.local` if it contains account-specific or sensitive values.

PART 6: ECR Build and Push from VS Code Terminal

Step 6.1: Confirm ECR Repositories Exist

```
aws ecr describe-repositories --region us-east-1
```

You should have:

```
flipkart-api-gateway
flipkart-user-service
flipkart-product-service
flipkart-cart-service
flipkart-order-service
flipkart-payment-service
flipkart-notification-service
flipkart-frontend
```

Step 6.2: Login to ECR

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin
ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com
```

Step 6.3: Build and Push One Service

Example for API Gateway:

```
cd api-gateway
docker build -t flipkart-api-gateway .
docker tag flipkart-api-gateway:latest ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/flipkart-api-
gateway:latest
docker push ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/flipkart-api-gateway:latest
cd ..
```

Step 6.4: Build and Push All Services with Template Script

The workspace includes a helper template:

```
scripts/aws-ecr-build-push-template.sh
```

Usage:

```
cp scripts/aws-env.example scripts/aws-env.local
# edit scripts/aws-env.local first
./scripts/aws-ecr-build-push-template.sh
```

This builds and pushes all 8 Docker images.

PART 7: Deploy New Images to ECS from VS Code

After images are pushed to ECR, force ECS to start new tasks:

```
aws ecs update-service \  
  --cluster flipkart-cluster \  
  --service api-gateway-service \  
  --force-new-deployment \  
  --region us-east-1
```

Repeat for each service.

The workspace includes a helper template:

```
scripts/aws-ecs-force-deploy-template.sh
```

Run:

```
./scripts/aws-ecs-force-deploy-template.sh
```

Then monitor:

```
aws ecs describe-services \  
  --cluster flipkart-cluster \  
  --services api-gateway-service \  
  --region us-east-1
```

In AWS Console:

```
ECS → Clusters → flipkart-cluster → Services
```

PART 8: Build Infrastructure from VS Code

You have three choices.

Choice A: Console-Only Infrastructure

Use the PDF:

`docs/AWS_CONSOLE_ONLY_SETUP_GUIDE.pdf`

This is best for learning AWS visually.

Choice B: Terraform from VS Code

The workspace has a Terraform starter:

`infrastructure/terraform`

Commands:

```
cd infrastructure/terraform
terraform init
terraform fmt
terraform validate
terraform plan -var='docdb_master_password=ChangeThisStrongPassword123!'
terraform apply -var='docdb_master_password=ChangeThisStrongPassword123!'
```

This creates baseline infrastructure like:

- VPC
- Public/private subnets
- ECS cluster
- ECR repositories
- ALB
- Security groups
- DocumentDB starter

Destroy lab infra when done:

```
terraform destroy -var='docdb_master_password=ChangeThisStrongPassword123!'
```

Warning: Terraform destroy deletes infrastructure. Use only in test/lab environments.

Choice C: CI/CD Builds Infrastructure or Deploys App

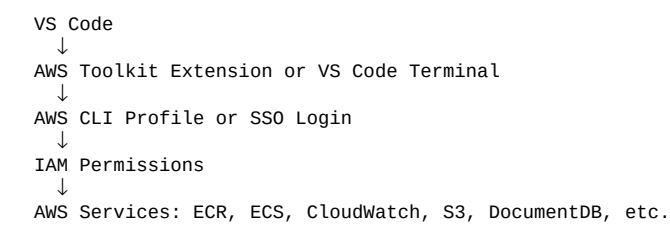
Use one of:

- AWS CodePipeline + CodeBuild
- GitHub Actions
- Jenkins on EC2

For most students/projects, GitHub Actions is easiest.

PART 9: How VS Code Links to Your AWS Infra

VS Code does not magically connect to AWS infrastructure by itself. It connects through these layers:



The important files are:

File	Purpose
.env	Local Docker Compose environment
scripts/aws-env.local	Local AWS account/deploy variables
docker-compose.yml	Local full-stack run
Dockerfile in each service	Image build instructions
infrastructure/terraform/*.tf	Infrastructure as code
.github/workflows/ci.yml	CI syntax checks

PART 10: Recommended Deployment Flow from VS Code

Use this flow after AWS infrastructure is already created:

1. Confirm local app works:

```
docker compose up --build
```

2. Build frontend:

```
cd frontend
npm install
npm run build
cd ..
```

3. Check backend syntax:

```
./scripts/check-backend-syntax.sh
```

4. Login to AWS:

```
aws sso login --profile flipkart-dev
# or use aws configure profile
```

5. Build and push images:

```
./scripts/aws-ecr-build-push-template.sh
```

6. Force ECS deployment:

```
./scripts/aws-ecs-force-deploy-template.sh
```

7. Watch deployment:

```
AWS Console → ECS → flipkart-cluster → Services
AWS Console → EC2 → Target Groups → Targets
AWS Console → CloudWatch → Logs
```

8. Test app:

```
curl http://ALB_DNS_NAME/api/health
curl http://ALB_DNS_NAME/api/products
```

PART 11: VS Code Tasks

The workspace includes VS Code task definitions in:

`.vscode/tasks.json`

Useful tasks:

- Docker Compose Up
- Docker Compose Down
- Seed Products
- Backend Syntax Check
- Frontend Build
- Terraform Init
- Terraform Validate
- Terraform Plan

Run tasks:

Terminal → Run Task

PART 12: GitHub Actions Connection

If code is pushed to GitHub, CI/CD can build and deploy automatically.

Required GitHub Secrets

Add in GitHub:

Settings → Secrets and variables → Actions

Secrets:

```
AWS_ACCESS_KEY_ID  
AWS_SECRET_ACCESS_KEY  
AWS_REGION=us-east-1  
AWS_ACCOUNT_ID
```

Better production approach:

- Use GitHub OIDC with IAM role instead of long-lived access keys.

Workflow Concept

```
Push code to GitHub  
↓  
GitHub Actions builds Docker image  
↓  
Pushes image to ECR  
↓  
Updates ECS service  
↓  
ECS starts new tasks
```


PART 13: CodePipeline Connection

If using AWS CodePipeline:

```
GitHub or CodeCommit
↓
CodePipeline Source Stage
↓
CodeBuild Docker Build
↓
ECR Push
↓
ECS Deploy
```

Use this if you want AWS-native CI/CD.

PART 14: Security Rules for VS Code and AWS

Never commit these:

- AWS access keys
- .env
- scripts/aws-env.local
- DocumentDB password
- JWT secret
- SMTP password
- Payment keys

Use:

- AWS Secrets Manager
- GitHub Actions secrets
- IAM Identity Center SSO
- IAM roles for ECS tasks

Check before commit:

```
git status
git diff --cached
```

PART 15: Common AWS CLI Commands from VS Code

Who am I?

```
aws sts get-caller-identity
```

List ECR repositories:

```
aws ecr describe-repositories --region us-east-1
```

List ECS clusters:

```
aws ecs list-clusters --region us-east-1
```

List ECS services:

```
aws ecs list-services --cluster flipkart-cluster --region us-east-1
```

Describe service:

```
aws ecs describe-services \
  --cluster flipkart-cluster \
  --services api-gateway-service \
  --region us-east-1
```

View recent CloudWatch log streams:

```
aws logs describe-log-streams \
  --log-group-name /ecs/flipkart-api-gateway \
  --order-by LastEventTime \
  --descending \
  --max-items 5 \
  --region us-east-1
```

PART 16: If You Want Fully Automated Infra + App Deployment

Recommended mature setup:

1. Terraform creates VPC, ECR, ECS, ALB, DocumentDB and security groups.
2. GitHub Actions builds Docker images and pushes to ECR.
3. GitHub Actions updates ECS services.
4. CloudWatch monitors and alerts.
5. Secrets Manager stores credentials.
6. Route 53 and ACM provide domain and HTTPS.

Workflow:

VS Code → Git commit → GitHub push → CI/CD → AWS ECS deployment

This is the cleanest long-term workflow.

PART 17: Final Checklist

- ☐ VS Code opened at project root.
- ☐ AWS Toolkit installed.
- ☐ AWS CLI configured with `flipkart-dev` profile.
- ☐ Docker is running.
- ☐ Local app runs with Docker Compose.
- ☐ ECR repositories exist.
- ☐ `scripts/aws-env.local` configured.
- ☐ Docker images pushed to ECR.
- ☐ ECS task definitions use ECR image URIs.
- ☐ ECS services running.
- ☐ ALB target groups healthy.
- ☐ CloudWatch logs visible.
- ☐ Application opens from ALB DNS or custom domain.